Final Quick-Reference Interview Cheat Sheet — WariMitra

Document Identifier: WM-CS-FINAL-031

Classification: High-Yield Pre-Interview Cheat Sheet & Rapid Revision Card

Target Audience: BA / Technical BA / Agile Product Owner Candidates (10-Minute Read Before Interviews)

1. Quick Facts & Elevator Pitch

- **Project Name:** WariMitra ("Digital Wari Ecosystem")
- **Domain:** GovTech / Mass Gathering Public Safety / Emergency Incident Response
- **Context & Scale:** 500,000 to 1,000,000+ pilgrims (*Varkaris*), 21 walking days, 250+ km corridor (Alandi/Pune to Pandharpur, Maharashtra).
- **The Core Problem:** Fragmented manual management—delayed medical dispatch (45–90 min), fatal crowd bottlenecks, missing persons taking 24–48 hours to find, and uncoordinated civic resources.
- **The Solution:** Cross-platform GovTech platform: Flutter mobile app (multilingual Marathi/English) for pilgrims and volunteers; Next.js 15 web command center for police and healthcare dispatchers; backed by Firebase Serverless (`asia-south1`) and an Ethereum Sepolia Web3 Trust Layer for zero-cost immutable audit trails.

2. Key Architecture & Technology Points

- **Frontend:** Next.js 16 (App Router), React 19, TypeScript, TailwindCSS 4, Zustand, `react-window` (virtualized incident feed), Google Maps JavaScript API.
- **Mobile:** Flutter 3.9.2, Dart, Riverpod, `geolocator`, `google_maps_flutter`, Firestore offline persistence cache.
- **Backend:** Firebase Cloud Functions (Node.js 20, TypeScript), Cloud Firestore NoSQL, Cloud Storage, Firebase Cloud Messaging (FCM).
- **Web3 Trust Layer:** Ethers.js v6, Ethereum Sepolia Testnet, 0-ETH asynchronous self-transactions embedding SHA-256 event snapshots in tx hex data.
- **Why Firebase over Django?** Django monolith struggled with database lockups under 50k+ live GPS

- **Why Firebase over Django?** Django monolith struggled with database lockups under 50k+ live GPS pings/min. Firebase auto-scales to 10k+ writes/sec, provides native real-time listeners for Next.js, and reduced projected operating costs to ~$185/month.

3. Top 5 Core Features & Technical Mechanics

1. **3-Second Hold Emergency SOS:** Physics-based gesture to prevent false alarms; client-side UUIDv4 idempotency key; server-side rate limit (5 alerts/min/user); India-only geofence bounds ($8-35^\circ\text{N}, 68-97.5^\circ\text{E}$).
2. **Geohash Medical Camp Radar:** `getMedicalCamps` callable Cloud Function computes Geohash bounding boxes for a 10 km radius; queries active camps; filters by Haversine distance; returns sorted camps with bed occupancy in 320ms p95.
3. **Command Center Incident Feed:** Real-time Firestore snapshot listeners; virtualized rendering via `react-window` maintaining 60 FPS under 1,000+ alerts; one-click status triage (`pending` $\rightarrow$ `acknowledged` $\rightarrow$ `dispatched` $\rightarrow$ `resolved`).
4. **Family Circle Tracking:** Ad-hoc group creation using unique 6-character invite codes; 30-second background location updates with battery level monitoring.
5. **Web3 Audit Notary:** Background Cloud Function triggers compute canonical SHA-256 hash snapshots of SOS state changes and anchors them to Sepolia testnet at ₹0 recurring gas cost.

4. Key Metrics (Empirical vs. Claimed)

- **SOS Creation Latency: 1.2s (p95)** — *VERIFIED* benchmark under load tests (`README.md:243`).
- **Firestore Read Latency: 320ms (p95)** — *VERIFIED* single and indexed queries (`README.md:244`).
- **Real-Time Client Update: 1.5s (p95)** — *VERIFIED* listener propagation delay (`README.md:245`).
- **Rate Limit Hard Cap: 5 alerts/min/user** — *VERIFIED* in Cloud Function code (`functions/src/index.ts:68`).
- **50% Emergency Response Reduction:** *CLAIMED STRATEGIC TARGET* — Modeled against assumed 45-minute manual baseline; honestly explain that pre-digital field dispatch logs do not exist in the repo.
- **Operating Cost: ~$185/month for 1M users** — *ESTIMATED* capacity planning model on Firebase

- **Operating Cost: ~$185/month for 1M users** — *ESTIMATED* capacity planning model on Firebase Blaze pricing.

5. Stakeholders & 7 RBAC Roles

- **7 Roles:** `pilgrim`, `volunteer`, `dindi_leader`, `medical_officer`, `police_officer`, `ngo`, `admin`.
- **Enforcement:** Custom User Claims in Firebase Auth (`auth.token.userType`) validated at database level via `firestore.rules`.
- **Key Stakeholder Alignment:** Balanced Police need for security (preventing false alarms) against Pilgrim need for speed by allowing friction-free SOS (phone OTP + rate limiting) while restricting status updates and dispatch to verified authority roles.

6. What Was My Contribution? (Interview Script)

> *"The project implements a complete GovTech mass-gathering platform across Flutter, Next.js, and Firebase Cloud Functions. As the Technical Business Analyst, I reverse-engineered business workflows, authored the BRD and FRD with Given-When-Then acceptance criteria, formulated the Geohash and anti-abuse validation rules, established the 100% bidirectional Requirements Traceability Matrix (RTM), and structured the IEEE 829 UAT acceptance plan."*

7. Fast Technical Q&A

- **How does the Web3 layer cost ₹0?** It sends 0-ETH transactions to itself on public testnet (Sepolia), embedding the SHA-256 hash into the transaction hex data payload. No custom smart contracts, no recurring gas fees.
- **What if blockchain RPC fails?** Fail-safe fallback catches the error and writes the hash to the local Firestore `blockchainProofs` collection with status `LOCAL_PROOF_ONLY`. Emergency dispatch is never blocked.
- **What is Little's Law in the queue feature?** $L = \lambda W$ (Average queue length = arrival rate $\times$ average wait time). Used as a deterministic mathematical heuristic to estimate temple queue wait times without expensive deep learning.